An Ethernet-Enabled Customizable Neural Inference Framework on Low-Cost FPGA SoC for High-Frequency Trading

Ahmet Batuhan Özkaner* Mustafa Alper Ilık Baykal Sarıoğlu

Department of Electrical and Electronics Engineering, Istanbul Bilgi University, Istanbul, Türkiye

Abstract

High-frequency trading requires not only accurate decisions but predictable, ultra-low latency, which general-purpose CPUs cannot guarantee under scheduling and memory-hierarchy jitter. This work presents an Ethernet-enabled, *customizable* neural-inference framework embedded entirely in the programmable logic of a low-cost Xilinx Zynq-7000 SoC. The processing system handles packet reception and control; a hardware multi-layer perceptron (40-64-32-16-3) performs inference in the fabric. Feature vectors are streamed over AXI-DMA / AXI-Stream and the model is deployed in $Q8.8$ fixed point, so a Python-trained network is re-embedded without redesigning the datapath. The design uses 87.3 % of the DSP slices and 27.2 % of the LUTs at an estimated 2.72 W, closes timing at 125 MHz, and completes an inference in 768 ns. A cycle counter read back after every inference shows this latency to be invariant: over 100000 inferences the standard deviation is zero. The same network run bare-metal on the SoC's own Cortex-A9 at `-O3` requires 52.4 μs, so the fabric path is 68 times faster and, unlike the processor, has no spread at all. Because the feature set, network shape, fixed-point format and decision classes are configurable, the system is a reusable FPGA-embedded inference framework, demonstrated on an HFT decision task.

Keywords: FPGA, Neural network inference, Low-latency systems, Fixed-point arithmetic, High-frequency trading

1. Introduction

In high-frequency trading, the value of a decision decays within microseconds, so the time between the arrival of market data and the emission of a trading action is as important as the correctness of the action itself. That this competition is economically consequential rather than merely technical has been argued at length: Budish et al. [1] model the resulting latency race as an equilibrium feature of continuous limit order books, and Aquilina et al. [2] quantify its cost from exchange message data, finding races decided on the scale of microseconds. CPU- and GPU-based pipelines insert several sources of non-deterministic delay between these two events: operating-system scheduling and interrupts, cache and memory hierarchy behavior, and the depth of the software stack. The result is not only higher mean latency but also a heavy-tailed distribution (jitter) that is unacceptable when worst-case response time must be bounded.

This paper argues that the inference itself should be moved off the processor and into the FPGA fabric, where the datapath is deterministic and the worst-case latency is governed by a fixed pipeline rather than by the operating system. We present a neural-inference engine that runs *inside* the programmable logic of a low-cost Zynq-7000 SoC, fed over Ethernet through the processing system and an AXI-DMA / AXI-Stream datapath. Crucially, the engine is not tied to one trained model: the input features, the MLP topology, the fixed-point format, and the output decision classes are all user-configurable, making the system a reusable FPGA-embedded AI framework rather than a single fixed accelerator. The framework is validated on a three-class directional decision task over Binance BTCUSDT trade data, specified in Section 6.3.

1.1. Main Contributions

The main contributions of this work are:

- A hardware MLP inference core for a low-cost Zynq-7000 SoC built on 192 DSP-based multipliers and a fabric adder tree in a 13-stage pipeline, whose latency follows an analytic model ($\lceil n_\ell/3 \rceil + 14$ cycles per layer) whose predicted total agrees exactly with the measured input-to-done interval and with an on-chip cycle counter read back on every inference, and whose per-layer predictions agree with the oscilloscope captures to within one clock cycle.
- Run-time reconfigurability of the network *shape*, not only of its weights: layer widths reside in configuration registers, so networks of different topologies are deployed into an unchanged bitstream over AXI-Lite without any synthesis step.
- A three-level verification methodology (an independent bit-accurate reference model, directed RTL simulation, and on-hardware comparison) which uncovered three pipeline-alignment defects invisible to synthesis and to end-to-end testing, and which after correction establishes bit-exact agreement over 400,000 inferences and 1,200,000 raw output values across four topologies.
- An Ethernet-fed hardware/software co-design with a per-stage latency breakdown measured over 10^6 packets, isolating how much of the end-to-end path is attributable to the deterministic hardware core and how much to the host and network software around it.
- Quantitative evidence for the determinism claim rather than illustrative captures: an on-chip cycle counter

read back with every inference shows a single-valued latency distribution over 100000 inferences, and a like-for-like software baseline on the SoC's own Cortex-A9, running identical fixed-point arithmetic, establishes the cost of the alternative.

2. Related Work

2.1. FPGA Acceleration in Financial Systems

Reconfigurable logic has been applied to electronic trading since the point at which software network stacks ceased to be competitive at microsecond timescales. Leber et al. [3] moved market-data feed handling and order generation into an FPGA and reported wire-to-wire latencies roughly an order of magnitude below a tuned software path, establishing the pattern that the determinism of a fixed datapath matters as much as its mean latency. Lockwood and Monga [4] pursued the same argument at data-center scale, placing the entire service in programmable logic to remove operating system and kernel-stack variability. More recent work has raised the level of abstraction rather than the level of ambition: Puranik et al. [5] synthesised a trading-system order-processing back end through a high-level synthesis flow, reporting roughly 500 ns per order against approximately 1 μs for the equivalent software, and Brown et al. [6] applied dataflow transformations to a credit-default-swap pricing engine on a data-center accelerator card.

What these designs share is that the accelerated function is fixed at elaboration time. A feed handler, an order book, a pricing kernel or a rule-based strategy is compiled into the fabric; changing the strategy means changing the bitstream. This is appropriate when the algorithm is stable, but it does not address the case in which the decision logic is a *learned* model that is periodically retrained.

2.2. Neural Network Inference on FPGAs

The FPGA deep-learning literature has largely converged on compiler-style frameworks that translate a trained network into a bitstream. Zhang et al. [7] formalised the design-space exploration underlying such accelerators; Qiu et al. [8] showed that the resulting designs fit within the resource envelope of an embedded Zynq part, and Angel-Eye [9] added a quantization flow and a compiler on top of that line of work; FINN [10] and its successor FINN-R [11] generate dataflow architectures for binarized and quantized networks, reaching classification latencies of a few hundred nanoseconds; and hls4ml [12] targets the same sub-microsecond regime for the trigger systems of particle-physics experiments, which resemble the trading problem in that a decision must be produced within a hard latency budget. SparkNoC [13] and comparable edge accelerators optimize energy efficiency for convolutional workloads, while Tsoutsouras et al. [14] show the same exploration methodology applied to a non-neural classifier on a Zynq SoC; Wang et al. [15] survey the field more broadly. On embedded SoC targets the movement of data between the processing system and the fabric is frequently the limiting factor rather than the arithmetic itself, as quantified for a CNN accelerator by Rios-Navarro et al. [16], an observation that the per-stage breakdown of Section 6 reproduces for the present design.

The arithmetic these designs rely on is well understood. Fixed-point and integer-only inference has been shown to preserve accuracy for a wide range of networks [17, 18], and the trade-offs are surveyed comprehensively by Gholami et al. [19]; the $Q8.8$ format adopted here sits at the conservative end of that spectrum.

The common property of this second group is that the network topology is baked into the generated hardware. FINN, FINN-R, Angel-Eye and hls4ml all require a new synthesis and implementation run when the network changes, which for a mid-size design is a multi-hour turnaround. They are also, without exception, evaluated as compute kernels: the input arrives from host memory or an on-chip stream, and the network interface is outside the scope of the reported latency.

2.3. Run-Time Programmable Overlays

A separate line of work removes the re-synthesis requirement by placing a fixed, programmable engine on the fabric and expressing the network as data rather than as logic. VTA [20] defines an instruction set for tensor operations and a compiler stack above it, so that a change of network is a change of instruction stream. OPU [21] and its lightweight successor [22] make the same argument specifically for convolutional networks on FPGAs, and state the property explicitly: the network is compiled to microcode that the overlay executes without reconfiguration. Commercial toolchains adopt the same model: the deep-learning processing unit shipped by AMD for Zynq and Versal parts [23] is a fixed overlay driven by compiled instructions, so that a new network is deployed by recompiling its instruction stream rather than by re-implementing the device.

The engine described in this paper belongs to this family, and we make no claim to have introduced run-time programmability. The distinction lies in the design point rather than in the mechanism. Published overlays are throughput architectures: they are dimensioned for convolutional workloads, evaluated in frames or images per second, and their reported latencies are per batch or per frame rather than per inference. Among published overlays that require no re-synthesis we are aware of none that reports single-inference latency below one microsecond, and the reason is structural rather than incidental: overlay evaluations are reported per frame or per batch, and the tiled dataflow that gives an overlay its generality implies that a network of this size occupies the engine for many passes. We state the claim in this restricted form because it rests on the absence of a reported figure rather than on a measurement of our own. The present design accepts a much narrower scope, fully connected layers within fixed bounds, in exchange for a latency figure that is a property of a single inference.

2.4. The Latency Extreme

At the opposite end of the trade-off, LogicNets [24] abandons arithmetic units altogether and compiles quantized neurons directly into lookup-table logic, reaching inference

latencies of the order of nanoseconds for networks comparable in size to the one considered here. Subsequent work in the same direction has extended the approach to higher-order functions, by learning piecewise polynomials inside each lookup table [25], and to deeper topologies, by hiding a multi-layer sub-network inside a single synthesizable Boolean function [26].

These results are one to two orders of magnitude faster than the 768 ns reported below, and we state that plainly rather than selecting a comparison that flatters the present work. The relevant question is what the difference buys. Compiling a network into LUT logic makes its weights part of the circuit: retraining requires a new synthesis, place and route, and bitstream. For a model that is retrained on a schedule, that turnaround is the binding constraint rather than the inference latency. The design space therefore has at least three distinguishable points, and the present work occupies the middle one: LUT-mapped networks give nanoseconds with per-model synthesis, overlays give re-programmability with per-frame throughput, and the engine described here gives sub-microsecond single-inference latency with re-programmability, on a low-cost part.

2.5. Learned Models for Order-Book Prediction

On the modelling side, deep networks applied to limit-order-book data are an established line of work. Zhang et al. [27] combine convolutional and recurrent layers for mid-price movement prediction, and the FI-2010 benchmark of Ntakaris et al. [28] provides a common evaluation basis. This literature is concerned with predictive quality and is typically evaluated offline; the deployment latency of the resulting model is rarely reported, and the models are generally too large for a microsecond decision budget without substantial compression.

2.6. Positioning of This Work

Taken together, the preceding sections describe a trade-off between how fast a single inference completes and how expensive it is to change the model. LUT-mapped networks sit at one end, compiler-generated dataflow accelerators in the middle, and overlays at the other end. What is missing from that spectrum, and what this paper contributes, is a point that combines sub-microsecond single-inference latency with re-programmability, on a part that costs a few hundred dollars, together with evidence that the deployed arithmetic is correct.

We therefore do not claim run-time reconfigurability as such, nor the lowest latency in the literature. The claim is the conjunction of four properties, none of which is individually novel but whose combination we have not found reported: (i) single-inference latency below one microsecond, given as an analytic function of the network shape and confirmed on hardware; (ii) deployment of networks of different topologies into an unchanged bitstream; (iii) an Ethernet to decision path measured end to end, with the contribution of each stage separated; and (iv) bit-exact agreement between the deployed arithmetic and an independent reference model over 4×10^5 inferences. The fourth is uncommon in this literature and, as Section 6.2 reports, was not a formality in our case.

Table 1 positions the work against representative prior designs. We note that the comparison of absolute latency figures across these rows is of limited value, since the reported quantities are not measured between the same two points: FINN and hls4ml report a compute-kernel latency, whereas the figure reported here is the end-to-end path from an Ethernet frame to a decision. The core latency of this design is given separately in Section 6 to make the comparison meaningful.

One comparison we do not report should be named rather than left implicit. We do not provide a like-for-like implementation of the same network generated by hls4ml or Vitis HLS on the same device, which would place a measured point at the compiler-based end of the design space described above instead of an argued one. The experiment is straightforward and we identify it as the most useful extension of this evaluation.

3. Proposed Framework

3.1. System Overview

The proposed system follows a strict hardware/software partition on a single Zynq SoC, illustrated in Fig. 1. Market data arrives over Ethernet and is received by the PS, which prepares the feature vector and issues it to the PL through an AXI-DMA engine. The custom `mlp_axi_wrapper` IP receives the stream, runs the inference core, and returns the decision and status through AXI-Lite. The PS therefore performs only I/O and control, while the entire inference workload is executed in the PL fabric. The end-to-end conceptual flow is:

$$\text{Ethernet} \rightarrow \text{PS} \rightarrow \text{AXI-DMA/Stream} \rightarrow \text{MLP core} \rightarrow \text{BUY/HOLD/SELL}.$$

3.2. Ethernet-Enabled Data Path

Feature packets are delivered over Ethernet to the PS, parsed, and forwarded to the PL. Each 32-bit AXI-Stream word carries two 16-bit $Q8.8$ values, and the 40 features, zero-padded to the 64-wide datapath, are transferred as 32 stream words on which the engine auto-starts. Every stage of this path is instrumented with PS-side timestamps and quantified over a 1,004,229-packet run in Section 6 (Table 12).

3.3. Customization Flow

The framework is reusable across models and strategies. A user trains an MLP on their own dataset and feature set; the trained parameters are quantized and the on-chip weight/bias memories are regenerated, while the datapath, AXI interfaces, and control logic are unchanged. Table 2 summarizes the configurable components.

Table 1: Comparison with representative FPGA-based financial-acceleration and neural-network inference studies. "Re-synth." indicates whether changing the deployed model requires a new bitstream. The reported metrics are not measured between the same two points and are given for orientation rather than as a ranking; see the discussion in the text.

Study	Platform	Function	Precision	Re-synth.	Net. I/O	Reported metric
Leber et al. [3]	Virtex-5	Feed handler, order gen.	n/a	Yes	Yes	2.6 μs half round trip
Puranik et al. [5]	Zynq	Order processing, HLS	n/a	Yes	No	$\sim$500 ns per order
Brown et al. [6]	Alveo U280	CDS pricing	Float	Yes	No	27,675 options/s
FINN [10]	ZC706	Binarised MLP/CNN	1 bit	Yes	No	0.31 μs, 12.3 M cls./s
hls4ml [12]	Virtex-7	MLP, trigger	Fixed	Yes	No	$\sim$100 ns, jet tagging
Angel-Eye [9]	Zynq XC7Z045	CNN compiler	8/16 bit	Yes	No	137 GOP/s, VGG16
SparkNoC [13]	Arria 10 GX1150	Lightweight CNN	Fixed	Yes	No	337.2 GOP/s
VTA [20]	Zynq, Ultra96	Tensor overlay	8 bit	No	No	Per-frame throughput
OPU [21]	Zynq XC7Z100	CNN overlay	8 bit	No	No	Per-frame throughput
LogicNets [24]	Virtex UltraScale+	LUT-mapped MLP	2 to 6 bit	Yes	No	$\sim$10 ns
This work	Zynq XC7Z020	MLP, run-time shape	16 bit	No	Yes	768 ns core, 68.6 μs end to end

Table 2: Customization options supported by the proposed framework

Component	Customizable Parameters
Input features	Feature vector contents and size (default 40)
MLP model	Retrainable weights/biases; hidden-layer sizes
Fixed-point format	Word/fraction split (default $Q8.8$)
Output decision	Class set, e.g. BUY / HOLD / SELL
Deployment flow	Automatic weight/bias memory regeneration

4. FPGA-Embedded Neural Inference Architecture

The embedded network is an MLP with topology $40 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 3$: a 40-dimensional input feature vector, three hidden layers with ReLU activation, and a 3-class output (BUY/HOLD/SELL); the argmax over the three raw scores is evaluated by the PS after read-back. Inference is performed entirely in hardware; no layer is offloaded to the PS.

4.1. Fixed-Point Quantization

The floating-point model is converted to a $Q8.8$ fixed-point representation (8 integer, 8 fractional bits), which lowers resource usage and enables single-cycle DSP multiplications while preserving accuracy. Weights and biases are stored in BRAM and addressed through the AXI-Lite configuration interface. Table 3 reports the effect of quantization on the reference model. The fidelity of the *hardware* implementation of this arithmetic is a separate question and is established independently in Section 6.2.

4.2. Parallel MAC-Based Inference Core

The inference core (`mlp_engine`) is built around 192 DSP slices used as parallel multipliers; the accumulation is not performed inside the DSP blocks but by a fabric adder tree, which is why the logic utilization is dominated by the reduction network rather than by the multiplications, organized as a 13-stage pipeline (Fig. 2): one address-issue stage (S0) followed by twelve datapath stages. Stages 1 and 2

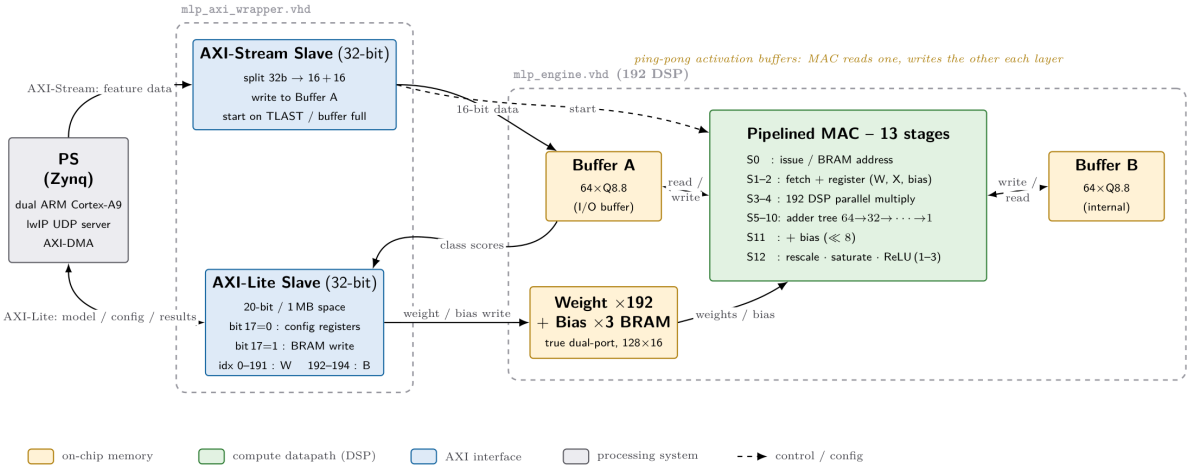


Figure 1: System architecture of the proposed framework on the Zynq-7000 SoC. Market data arrives over Ethernet at the PS, which forwards the prepared feature vector to the custom `mlp_axi_wrapper` core through AXI-DMA / AXI-Stream; the model parameters are loaded and the final decision is read back over AXI-Lite. The entire inference is executed by the 13-stage pipelined MAC engine in the PL.

Table 3: Effect of $Q8.8$ quantization on the reference model

Metric	Value
Float accuracy	61.74%
$Q8.8$ accuracy	61.65%
$Q8.8$ vs. float agreement	99.93% (macro F1 0.990)
Class change ratio (float vs. $Q8.8$)	0.074%

read weights/inputs from BRAM and register them; stages 3 and 4 perform the 192 parallel multiplications; stages 5 to 10 reduce the products through a balanced $64 \rightarrow 1$ adder tree; stage 11 adds the bias; and stage 12 rescales the result and applies the ReLU activation before write-back. Two ping-pong buffers (Buffer A / Buffer B) hold the inter-layer activations so that successive layers reuse the same MAC array.

5. Implementation on Zynq-7000 SoC

5.1. Hardware/Software Partitioning

Table 4 lists where each function is implemented. The PL implements the entire inference datapath and the measurement signals; the PS is responsible for the network interface, for handing the received feature vector to the DMA engine, and for high-level control.

Two aspects of this partition should be stated plainly, because they bound what the reported latency covers. First, feature construction and the conversion to $Q8.8$ are performed *upstream of the board*: the packet arriving at the PS already contains a quantized, zero-padded vector, and the PS copies it into a DMA-aligned buffer without transforming it. Moving feature construction onto the PS would add work to the measured path and is not claimed here. Second, the argmax over the three output scores is evaluated in PS software after the scores have been read back over AXI-Lite; it is a three-way comparison and is included in the read-out stage of the latency breakdown (Table 12) rather than in the core latency. Moving it into the PL would remove two of the three AXI-Lite reads and is a straightforward extension.

Table 4: Hardware/software partitioning of the proposed system

Component	Implementation Location
Ethernet packet reception	PS (software)
Feature vector hand-off	PS (software, `memcpy`)
Feature vector transfer	AXI-DMA (PS→PL)
MLP inference	PL (hardware)
Argmax decision	PS (software, after read-back)
Status/result registers	AXI-Lite (PS↔PL)
Debug signals	PL output pins

5.2. AXI-Stream and AXI-DMA Interface

The PS prepares the feature vector and transfers it to the PL with AXI-DMA; the `mlp_axi_wrapper` consumes it as an AXI-Stream slave, splitting each 32-bit word into two $Q8.8$ operands and starting the engine once a full vector is received. Control and status (start, busy, done, result) use AXI-Lite. This keeps the inference pipeline continuously fed without per-operand PS intervention.

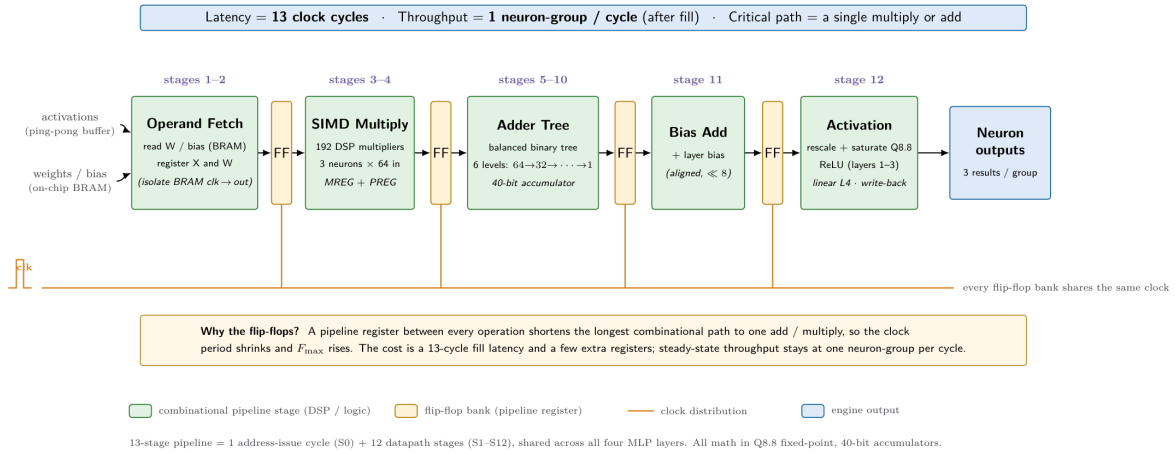


Figure 2: Pipeline datapath of the hardware MLP core (`mlp_engine`). After the address-issue stage (S0), stages 1 and 2 fetch the weights, inputs, and bias from BRAM; stages 3 and 4 perform the 192 parallel $Q8.8$ multiplications on the DSP slices; stages 5 to 10 form the balanced $64 \rightarrow 1$ adder tree; stage 11 adds the bias; and stage 12 rescales and applies the ReLU before the result is written back to the ping-pong activation buffers. The `mlp_axi_wrapper` splits each 32-bit AXI-Stream word into two 16-bit $Q8.8$ values and triggers the engine; weights/biases reside in BRAM and are loaded via AXI-Lite (address bit 17 selects configuration vs. memory).

5.3. Debug and Measurement Infrastructure

To measure hardware timing directly, the core exports the dedicated debug signals of Table 5. `dbg_data_in_pulse` marks the arrival of a complete feature vector, `dbg_layer1_done` to `dbg_layer4_done` pulse as each layer finishes, and `dbg_mlp_busy` brackets the whole inference. These pins are probed with an oscilloscope, so the per-layer latency is observed on real silicon rather than estimated (Section 6).

Table 5: Debug signals exported for hardware latency measurement. All signal names carry the prefix `dbg_`, omitted here for space.

Signal	Purpose
`data_in_pulse`	Full feature vector received
`layer1_done`	Layer 1 ($40 \rightarrow 64$) complete
`layer2_done`	Layer 2 ($64 \rightarrow 32$) complete
`layer3_done`	Layer 3 ($32 \rightarrow 16$) complete
`layer4_done`	Layer 4 ($16 \rightarrow 3$) complete
`mlp_busy`	Inference in progress; decision valid on the falling edge

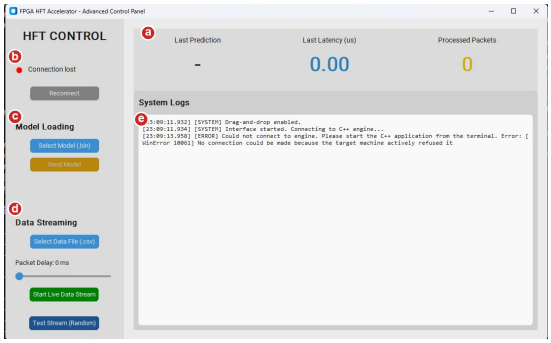


Figure 3: Host control application: (a) live decision, round-trip latency and packet count, (b) engine connection state, (c) model upload, (d) stream source and inter-packet delay, (e) event log. Deploying a different network is the single action in panel (c); no synthesis step is involved.

6. Experimental Evaluation

6.1. Experimental Setup

The framework is implemented on a Xilinx Zynq-7000 SoC. A host control application (Fig. 3) connects to the C++ host engine, uploads the quantized model, and streams either real CSV market data or randomized test packets while reporting the last prediction, last latency, and processed-packet count. Table 6 summarizes the setup.

Two distinct runs are reported in what follows and should not be conflated. The first measures the host round trip: the host timestamps a request before transmission and after the reply arrives, over 120,883 requests, and is the basis of Table 11 and of the host comparison in Section 6.6. The second runs the PS-side instrumentation, in which the firmware timestamps each stage of the on-board path with the Global Timer and returns the timestamps in the reply, over 1,004,229 packets, and is the basis of Table 12. The two give slightly different end-to-end figures (a median of 66.3 μs against 68.5 μs) because the instrumented run carries five additional timestamps and a larger reply payload, and because the runs were taken at different times with different host load. The difference is small relative to the host-side variability characterized in Section 6.6, and no claim in

Table 6: Experimental setup

Item	Description
FPGA device	Xilinx XC7Z020-1CLG484 (ZedBoard)
Design language	VHDL (PL); C++/Python (PS/host)
Tool version	Xilinx Vivado / Vitis 2025.2
PS software	Bare-metal, lwIP raw API (no OS on the board)
Data path	Ethernet $\rightarrow$ PS $\rightarrow$ AXI-DMA $\rightarrow$ AXI-Stream $\rightarrow$ PL
Inference core	MLP 40-64-32-16-3, $Q8.8$, 192 DSP MACs
Dataset	Binance BTCUSDT aggregated trades, 16631211 records, 2025-05-01 to 2025-05-18
Task	3-class direction, horizon 50 trades, $\theta = 5 \times 10^{-5}$
Validation split	Final 10 % chronologically, 1663113 samples
Host round-trip run	120,883 requests (Linux, host-timestamped)
Instrumented run	1,004,229 packets (PS-timestamped, per stage)
Clock frequency	125 MHz (PL)

this paper depends on it; where a single end-to-end number is quoted we use the instrumented run, since it is the one whose composition is known.

6.2. *Functional Verification of the Inference Core*

Reporting latency and resource figures for an accelerator is only meaningful if the accelerator is first shown to compute the intended function. We therefore verify the core at three levels before reporting any performance result.

Golden reference model. A bit-accurate software model of the datapath was written independently of the RTL. It reproduces every arithmetic detail of the deployed pipeline: the host $Q8.8$ conversion with saturation, the truncating weight conversion performed by the on-board loader, the 40-bit accumulation, the arithmetic right shift, the 16-bit saturation, the ReLU on all but the output layer, and the argmax rule used by the decision logic. A second, cycle-accurate model of `mlp_engine` was also written; it reproduces the issue FSM, the 13-stage token pipeline, the one-cycle BRAM read latency and the ping-pong write-back, and it is used to reason about pipeline alignment independently of the simulator.

Directed RTL simulation. Three directed tests exercise mutually independent mechanisms of the engine: a pass-through chain that isolates the multiply, reduce and rescale datapath, a bias-only chain that isolates the bias pipeline, and a multi-neuron test that isolates the write-back indexing. Each test writes its parameters through the same AXI-Lite address map used by the on-board loader, streams a stimulus vector over AXI-Stream, and compares the three read-back outputs against values computed by hand. Input positions that are not overwritten by the network are pre-loaded with distinctive values, so that a missing write-back cannot be mistaken for a correct result.

This step exposed three pipeline-alignment defects in the original design that were invisible to synthesis and to end-to-end testing (Table 7). All three are one-line index corrections; none changes the pipeline depth, the resource usage or the timing closure of the design. After correction, the cycle-accurate model matched the golden reference on 240 randomly generated cases (60 random parameter sets $\times$ 4 random input vectors), whereas the original alignment matched on none.

Table 7: Pipeline-alignment defects found by directed simulation

Signal	Stage	Consequence
Bias pipeline	$11 \rightarrow 9$	Each neuron received the bias of a neuron six positions earlier
Write-back token	$12 \rightarrow 11$	Results were written to the wrong activation indices
Input multiplexer	$2 \rightarrow 1$	Each layer read from the wrong ping-pong buffer

On-hardware bit-exact verification. The corrected design was verified on the board against the golden model over a stimulus set constructed to exercise the saturation, ReLU, sign-transition and zero-padding paths, rather than over recorded market data. The reason is that the features of Section 6.3 are clipped to $[-5, 5]$ and therefore occupy a narrow band near the center of the $Q8.8$ interval: replayed market data never drives the accumulator toward saturation and never exercises the clamping logic, so it cannot distinguish a correct datapath from one whose saturation path is broken. A verification stimulus has to reach the corners that the application itself does not. The firmware was extended to return the three raw 16-bit output values alongside the decision, so that the comparison is made on the arithmetic itself and not only on the argmax.

Table 8 reports the result. Across four networks the hardware reproduced the golden model on 400,000 inferences and on all 1,200,000 raw output values, without a single discrepancy. We note that agreement on the decision alone would be a weak check when the output distribution is concentrated on one class; the per-output comparison is reported for this reason.

6.3. *Dataset and Classification Task*

The workload used throughout this paper is a three-way directional classification on Binance BTCUSDT aggregated

Table 8: On-hardware bit-exact verification against the golden model. All four networks were loaded at run time into the *same* bitstream. The last column compares the three raw 16-bit outputs of every inference, i.e. 1,200,000 individual comparisons in total, none of which differed.

Network	Inferences	Decision	Outputs
40-64-32-16-3	100,000	100.0000%	100.0000%
40-64-64-32-3	100,000	100.0000%	100.0000%
40-32-16-8-3	100,000	100.0000%	100.0000%
40-10-16-8-3	100,000	100.0000%	100.0000%
Total	400,000	100.0000%	100.0000%

trades. The data are the public aggregated-trade archive for the instrument, covering 16631211 trades between 2025-05-01 00:00 UTC and 2025-05-18 00:44 UTC, approximately seventeen days of continuous tick data. Each aggregated trade record carries a price, a quantity and a taker-side flag.

The 40 features are derived from a trailing window of 40 trades and are dimensionless by construction, which is what makes a $Q8.8$ datapath appropriate. They comprise 20 slots holding successive relative price changes scaled by 1000, 10 trade quantities each divided by the mean quantity of the same window, and 10 taker-side indicators taking the values $+1$ and -1. Only 19 of the 20 price-change slots carry data; the remaining one is a constant zero, retained from an implementation in which the difference series was zero-padded to preserve length. It carries no information, and we report it rather than defend it as a design choice. Every feature is clipped to $[-5, 5]$, so the integer part of $Q8.8$ carries the dynamic range of the input with more than four bits of headroom, and the eight fractional bits determine the resolution.

The label is defined on a forward horizon of 50 trades. Writing P_t for the price of trade t and $r_t = (P_{t+50} - P_t)/P_t$, the class is *buy* for $r_t > \theta$, *sell* for $r_t < -\theta$ and *hold* otherwise, with $\theta = 5 \times 10^{-5}$. The threshold is a design choice that sets the balance of the problem, and it was selected because the two alternatives examined, 10^{-4} and 2×10^{-4}, leave the minority classes at 5 % and below 0.1 % of the validation set respectively, at which point the task degenerates into predicting the majority class.

The split is temporal and not random: the first 90 % of the chronologically ordered samples are used for training and the final 10 %, 1663113 samples, for validation. No sample in the validation period is available to the model at training time, so the reported figures are not affected by look-ahead leakage. No embargo was applied at the boundary, so the labels of the last 50 training samples are formed from prices that fall inside the validation period; at 50 samples out of 1663113 the effect is not measurable, but we note it rather than leave it to be inferred. The validation period contains 57.16 % hold, 21.50 % buy and 21.35 % sell, which fixes the majority-class baseline at 57.16 %. That figure, and not 33 %, is the reference against which the accuracies below must be read.

6.4. Model Accuracy and Classification Performance

Table 10 reports, for five trained networks, the accuracy in floating point under PyTorch, the accuracy of the same weights through the fixed-point datapath, and the margin over the majority-class baseline established above.

Two properties of the table are worth separating. The first concerns the arithmetic: floating point and $Q8.8$ agree to within 0.22 percentage points across all five networks and to within 0.1 percentage points for the reference network, so reducing the datapath to 16-bit fixed point does not materially change the decisions the network makes. The second concerns the task: the reference network exceeds the majority-class baseline by 4.58 percentage points on 1663113 held-out samples, whereas the two smallest networks do not. At this sample size the standard error of a proportion is 0.04 percentage points, so the margins of the three larger networks are two orders of magnitude beyond sampling noise, while network D's margin of 0.06 points is not distinguishable from zero and network E is measurably below the baseline. The classification problem is therefore learnable at this feature set but only above a capacity threshold, and we report the two networks that fail as readily as the three that succeed, since a table containing only the successful configurations would misrepresent how narrow the margin is.

The fixed-point column is computed with a bit-accurate model of the deployed datapath rather than by streaming 1663113 samples through the board, which would take hours over UDP. This is legitimate here for a specific reason: Section 6.2 establishes that the hardware and that model agree bit for bit, on every decision and on every raw output value, over 400000 inferences across four topologies. The fixed-point column is therefore a hardware result in the sense that matters, and the verification section is what licenses reading it that way.

Table 9 gives the per-class behavior of the reference network, which the aggregate accuracy conceals. Recall is roughly balanced across the three classes, between 0.59 and 0.64, so the network is not simply defaulting to the majority class; precision, however, is markedly lower on the two minority classes, around 0.51 to 0.57 against 0.70 for hold. In trading terms the network identifies most directional moves but pays for it with false positives, which is the expected shape for a threshold this small.

Predictive quality is not a contribution of this work, and no claim is made that this network would be profitable after transaction costs. It is reported here so that the workload the hardware executes is fully specified, and so that the fixed-point agreement in Table 10 is anchored to a task whose difficulty the reader can judge.

6.5. System-Level Packet-to-Decision Latency

Table 11 reports the measured end-to-end packet-to-decision latency of the full FPGA system on the Linux host over 120,883 inference requests. Each request is a UDP round trip from the host to the board and back, so it includes the network, the PS software, the AXI-DMA transfer, and the PL core. The median is 66.3 μs; the pure hardware core

Table 9: Per-class performance of the reference network ($40 \to 64 \to 32 \to 16 \to 3$) on the 1663113-sample validation period

Class	Share	Recall	Precision	F_1
Hold	57.16%	0.619	0.698	0.656
Buy	21.50%	0.595	0.570	0.582
Sell	21.35%	0.637	0.506	0.564
Macro average		0.617	0.591	0.601

Table 10: Floating-point against $Q8.8$ accuracy on the 1663113-sample validation period, and the margin over the 57.16 % majority-class baseline. Model A is the reference network used throughout the paper.

Model	Topology	Float (%)	$Q8.8$ (%)	$\lvert\Delta\rvert$ (pp)	vs. base (pp)
A	40-64-32-16-3	61.74	61.65	0.09	+4.58
C	40-64-48-32-3	60.62	60.54	0.08	+3.46
B	40-64-64-32-3	60.58	60.59	0.02	+3.42
D	40-32-16-8-3	57.10	56.88	0.22	−0.06
E	40-10-16-8-3	56.86	57.01	0.15	−0.30

(Section 6, 768 ns) is therefore about 1 % of the end-to-end time, confirming that the remaining cost is dominated by the operating system, the network, and the PS software rather than by the inference itself.

One feature of the table needs comment, since it looks like an error and is not. The mean, 65.6 μs, lies below the median, 66.3 μs, even though the maximum is 3275.9 μs. The distribution is left-skewed in its body and only sparsely populated in its right tail: the first percentile is at 52.9 μs and the first quartile at 63.4 μs, so the median is 2.9 μs above the quartile below it but only 1.1 μs below the quartile above it. Just under 10 % of requests complete below 60 μs, and that lower shoulder moves the mean down further than the few extreme outliers move it up. The shoulder has the same origin as the bimodality of the reception stage in Table 12: a frame that arrives just before the polling loop reads the interface is serviced immediately, and one that arrives just after waits for the next pass.

Table 11: Measured end-to-end packet-to-decision latency of the FPGA system (Linux host, 120,883 requests)

Metric	Latency
Minimum	50.0 μs
Median	66.3 μs
Mean	65.6 μs
99th Percentile	79.6 μs
99.9th Percentile	109.7 μs
Maximum	3275.9 μs

6.6. How Much of the End-to-End Latency Belongs to the Host

The same UDP round-trip run was repeated on a Linux and a Windows host with the board, the bitstream and the model held fixed, which bounds how much of the end-to-end figure is determined by software outside the board. The Linux host completes a median request in 66.3 μs (p99 79.6 μs); the Windows host needs 476.3 μs (p99 608.7 μs, p99.9 1649 μs) and shows occasional timeouts. A factor of roughly seven in the median, and the entire heavy tail, therefore originates on the host side of the link rather than in the accelerator. This is also why the end-to-end figures in Table 11 should not be read as a property of the core. This experiment is not a software baseline for the inference itself, since the network is executed in the fabric in both runs; that comparison is given in Section 6.10.

6.7. Latency Breakdown

Because each layer exposes a done pulse, the inference can be decomposed on real hardware. Probing the debug pins with an oscilloscope yields the per-layer inter-pulse times of Fig. 4 and the total input-to-final-done interval of Fig. 5. Table 13 converts these to clock cycles at 125 MHz. The measured per-layer times are 280, 200, 160, and 118 ns for layers 1 to 4, and the measured end-to-end core interval is 768 ns.

These values are not merely observed; they follow from the architecture. The issue FSM emits one token per cycle while a layer is being issued, and each token carries three neurons, so a layer of n_ℓ outputs occupies $\lceil n_\ell/3 \rceil$ issue cycles. The FSM then waits for the pipeline to empty before advancing, which costs the full pipeline depth. The per-layer occupancy is therefore

$$c_\ell \;=\; \left\lceil \frac{n_\ell}{3} \right\rceil + D + 1, \qquad D = 13, \tag{1}$$

where D is the depth of the token pipeline. Table 13 compares this prediction with the oscilloscope measurement. The predicted total, $\sum_\ell c_\ell = 96$ cycles or 768 ns, agrees exactly with the measured input-to-done interval. Two of the four individual intervals read slightly below the prediction, layer 1 at 280 ns against a predicted 288 ns and layer 4 at 118 ns against 120 ns, so the four inter-pulse intervals sum to 758 ns where the interval measured directly between the first and last pulse is 768 ns. Both deviations are in the same direction and neither exceeds one clock period. The per-layer figures come from four separate captures in which one clock period occupies a small fraction of a division, and a spread of 10 ns accumulated over four such readings is consistent with that reading uncertainty. We therefore treat the total, which is corroborated independently by the cycle counter of Section 6.8, as the authoritative figure, and the per-layer captures as confirming the model to within one cycle. Latching a counter at each `layer_done` would remove the ambiguity entirely and is the appropriate instrument for per-layer timing at this resolution.

We note that the drain term in (1) is not negligible: of the 96 cycles, 56 are spent draining the pipeline between

layers rather than issuing work, so only 42 % of the core latency performs useful issue. This is a direct consequence of the ping-pong buffering, since a layer cannot begin reading a buffer until the preceding layer has finished writing it. For the narrow output layers the effect is pronounced: layer 4 issues a single token and then drains for 13 cycles. Overlapping the drain of one layer with the issue of the next would require double-buffering the activations at sub-layer granularity and is left as future work; at the present operating point the core latency is in any case a small fraction of the end-to-end path.

Arithmetic efficiency. The consequences for throughput are worth stating explicitly, since related work is frequently compared on this basis. With 192 multipliers at 125 MHz the peak arithmetic rate is 48 GOP/s. A single inference of the reference network performs $40{\times}64+64{\times}32+32{\times}16+16\times3 = 5168$ useful multiply-accumulate operations, but the zero-padded SIMD array issues 7680 multiplications, so 67 % of the issued arithmetic is useful. Combined with the drain overhead, the achieved rate is $2 \times 5168/768\,ns \approx 13.5\,GOP/s$, or 28 % of peak. The core sustains 1.30 M inferences per second; at system level the AXI-Stream wrapper does not overlap the reception of one feature vector with the computation of the previous one, which reduces the sustained rate to approximately 0.77 M inferences per second. Both figures are far in excess of what the host-side path can supply (Section 6), which is why the design is latency-optimized rather than throughput-optimized.

The full end-to-end path was then instrumented with PS-side timestamps over 1,004,229 packets (Table 12). The result isolates where the time is spent: of the 68.6 μs mean host-to-decision latency, only 14.4 μs is spent on the board (PS + PL), and the remaining 54 μs is host-side NIC, operating-system, and wire time. On the board, the Ethernet reception (EMAC + lwIP) dominates and is bimodal (p50 2.6 μs vs p99 14.2 μs), reflecting the phase of the polling loop relative to packet arrival, whereas the DMA, PL compute, and readout stages are tightly bounded (sub-microsecond to a few microseconds, with p99 within a few percent of the median). This confirms the central claim: the deterministic hardware core is a negligible and jitter-free fraction of the end-to-end latency, and the remaining cost is host- and network-side software, which a hardware Ethernet MAC in the PL would remove.

6.8. Determinism of the Hardware Core

The claim that the core is deterministic has so far rested on oscilloscope captures, which show one inference at a time and cannot exclude rare outliers. To establish it over a population, a 16-bit cycle counter was added to the engine. It is cleared when `start` is accepted, increments on every cycle in which the controller is not idle, and is latched when the final layer asserts `done`. The latched value is exposed at AXI-Lite offset `0x10` and read back by the firmware after every inference, so each logged decision carries the exact cycle count that produced it.

Over 100000 consecutive inferences driven at line rate the counter returned 95 on every inference. Minimum, maxi-

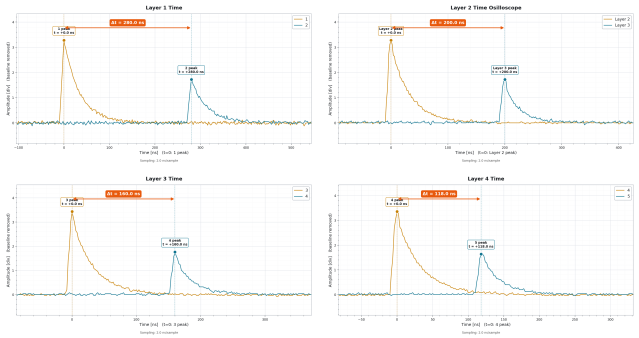


Figure 4: Oscilloscope captures of consecutive layer-done debug pulses; each inter-pulse interval directly measures one layer's processing time. Top-left to bottom-right: layer 1 (280 ns), layer 2 (200 ns), layer 3 (160 ns), layer 4 (118 ns).

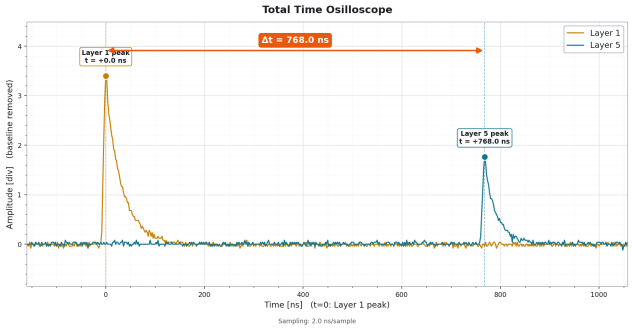


Figure 5: Oscilloscope capture of the first and last debug pulses of an inference, measuring the total hardware-core interval ($\Delta t = 768\,ns$).

Table 12: Latency breakdown from Ethernet packet to trading decision, over a 1,004,229-packet run (all values in μs). Percentiles on the subtotal row are computed over the per-packet sums, not by adding the per-stage percentiles; the two differ because a packet slow in one stage is not in general slow in the others (adding the per-stage p99 values would give 21.51 rather than 21.03).

Stage	Mean	p50 / p99	Measurement Method
Ethernet packet reception (EMAC + lwIP)	7.59	2.57 / 14.18	PS timestamping (bimodal)
Packet parsing and buffer hand-off	0.70	0.69 / 0.86	PS timestamping
AXI-DMA transfer to PL	3.70	3.70 / 3.78	PS/PL counters
AXI-Stream receive + MLP core (PL)	1.46	1.46 / 1.72	`dbg` pins (core 768 ns)
Result readout	0.96	0.96 / 0.97	AXI-Lite
Board subtotal (PS + PL)	14.40	9.41 / 21.03	On-board sum
Host end-to-end	68.56	68.50 / 80.92	Host round trip

Table 13: Per-layer latency of the MLP core at 125 MHz: analytic prediction from (1) against oscilloscope measurement on the layer-done debug pins. Layers 1 to 4 are $40 \to 64$, $64 \to 32$, $32 \to 16$ and $16 \to 3$ respectively.

			Predicted		Meas.
Layer	n_ℓ	$\lceil n_\ell/3 \rceil$	cyc	ns	ns
1	64	22	36	288	280
2	32	11	25	200	200
3	16	6	20	160	160
4	3	1	15	120	118
Total			96	768	768

mum and mean are all 95 and the standard deviation is zero; the distribution has a single point of support. The core therefore shows no jitter at all rather than a short tail: inference k and inference $k+1$ occupy the same number of cycles irrespective of the input values, of the activation pattern, or of concurrent AXI-Lite and DMA traffic. This is the property the application requires, since a trading system is exposed to the worst case and not to the mean.

The counter reads 95 where (1) predicts 96 because the two count from different points. The counter increments only in non-idle cycles and is latched on the same clock edge that returns the controller to the idle state, so it excludes the cycle in which `start` is sampled; adding that cycle gives 96, in agreement with both the analytic model and the 768 ns interval measured on the debug pins (Fig. 5). The difference is a property of the instrument rather than of the datapath, and the start-to-done convention is used throughout.

6.9. Comparison Against a Serial Baseline on the Same Device

The oscilloscope measurements establish that the hardware inference completes in 768 ns, deterministically. Interpreting that number requires a baseline, and the comparisons available in the literature are awkward: the figures reported by other accelerators are measured between different points, on different devices, for different networks. We therefore use a baseline that removes all of these confounds: an earlier version of this same design, on the same board, running the same network.

That earlier implementation is preserved in the version history of the artifact and can therefore be characterized exactly rather than from recollection. It used a sequential finite-state-machine controller driving a single multiply-accumulate unit, with the same network dimensions hard-wired into the controller. Its cycle count follows directly from the state machine: each neuron costs one cycle to load its bias, one cycle per input, and three further cycles to apply the activation, write the result and advance, so a layer of n_{out} neurons with n_{in} inputs occupies $n_{\text{out}}(n_{\text{in}} + 4)$ cycles. Summed over the four layers, with the layer transitions and the hardware argmax included, the design required 5639 cycles per inference.

The comparison must be stated carefully, because the two designs did not run at the same frequency: the sequential version was clocked at 100 MHz, giving 56.4 μs per inference, whereas the present design closes timing at 125 MHz. Two speed-up figures follow, and they answer different questions. Evaluated at a common 125 MHz the sequential design would require 45.1 μs, so the architectural change alone accounts for a factor of 58.7. As actually deployed, 56.4 μs then against 768 ns now, the improvement is a factor of 73.4, of which the remainder is the higher clock. We quote the former where the architecture is the subject.

Two mechanisms produce the gain, in unequal measure. Replacing the single MAC unit with 192 parallel multipliers removes the serialization of the arithmetic, and replacing per-operand AXI-Lite writes with a DMA-fed stream removes a processor transaction per input value. The second effect dominates: in the sequential design the cost scaled with the 5168 multiply-accumulate operations of the network rather than with its depth.

This is the most defensible speed-up figure in the paper, because it is derived rather than measured under conditions we no longer control: the device, the network and the measurement points are identical, the cycle count is recoverable from the archived source, and the one confound that remains, the clock, is separated out explicitly. The cross-platform comparisons of Table 1 deserve correspondingly more caution.

6.10. Software Inference on the Same SoC

The premise of the design is that moving inference off the processor and into the fabric is worth the effort, and on this device that premise can be tested directly, since the Zynq-7000 carries a 667 MHz dual-core Cortex-A9 alongside the fabric. The same network was therefore implemented in C and run bare-metal on the PS using the same $Q8.8$ arithmetic as the hardware: the same 40-bit accumulation, the same arithmetic right shift, the same saturation, the same ReLU and the same argmax rule, with weights converted by the same routine, so that the two paths compute identical values and differ only in how the arithmetic is scheduled. Latency was measured with the Zynq Global Timer over 1000 inferences with warm caches and with no network or DMA activity inside the measured region. Each inference was additionally timed individually so that the distribution, and not only the mean, could be compared against the hardware.

The software path requires 52.4 μs per inference on average against 768 ns in the fabric, a factor of 68. Compilation settings matter here and are worth stating explicitly: the identical source built without optimization requires 302 μs, so a baseline left at the default debug settings would have overstated the speed-up by almost a factor of six. The figure reported here is compiled at `-O3` and runs on a single core with no operating system. It is scalar code: the inner product is a plain loop over 16-bit operands and NEON intrinsics were not hand-written. At 667 MHz the measured time corresponds to about 34900 processor cycles for the 6704 multiply-accumulate operations the routine performs, or 5.2 cycles per operation, which is the cost of an unvectorized 16-bit loop on an in-order Cortex-A9. A hand-vectorized implementation would narrow the gap; we do not estimate by how much, since the rest of this paper reports measured rather than assumed quantities, and we record the absence of a NEON baseline as a limitation.

The distributions are more instructive than the means. The software path ranges from 51.2 μs at the minimum to 67.1 μs at the maximum, with a median of 52.0 μs and a 99th percentile of 52.6 μs. The spread between best and worst case is therefore 15.9 μs, which is more than twenty times the entire hardware inference, and the gap between the median and the 99th percentile alone is 570 ns, comparable to the whole of it. This is measured bare-metal, on a single core, with warm caches, with no operating system and with no network activity, which is to say under conditions more favorable than any deployment would provide; the residual variation is attributable to memory-system effects that software cannot control. The hardware path, by the counter of Section 6.8, has no spread at all. The comparison is therefore not only that the fabric is 68 times faster on average, but that its worst case equals its best case whereas the processor's worst case exceeds its best by a third of its own median.

A smaller observation points the same way. Running the identical routine with the trained weights rather than with a synthetic set changes the mean by about two percent, because the ReLU comparison is a data-dependent branch. The hardware has no counterpart to this: the pipeline evaluates the activation unconditionally, so the cycle count cannot depend on the values flowing through it.

One further comparison is worth drawing, because it sharpens what the contribution actually is. The sequential hardware design of Section 6.9 required 56.4 μs as deployed, which is slightly slower than optimized software on the same chip's processor. Moving the computation into the fabric was therefore not by itself sufficient; a serial datapath in logic bought nothing over a serial datapath in software. The gain reported here comes from the parallel issue structure and from removing the per-operand processor transactions, not from the change of substrate.

The comparison also has a dimension that latency alone does not express. The software path occupies the processor for the whole 52.4 μs, and during that interval it cannot service the network stack, whereas the hardware path returns the processor to its polling loop as soon as the DMA descriptor is posted. In a system whose residual latency is dominated by packet reception (Table 12), that difference compounds.

6.11. FPGA Resource Utilization

Table 14 reports the post-implementation resource usage on the XC7Z020, obtained from the post-implementation reports of the verified design. The core is DSP-bound (87.3 %); logic usage remains low (27.2 % of LUTs, 10.7 % of flip-flops), and 70.4 % of the block-RAM tiles are occupied.

The block-RAM figure requires interpretation, because it does not indicate that the memories are nearly full. As described in Section 6.13, the engine instantiates one memory per (neuron-slot, input) pair. Each of these 192 memories holds 128 words of 16 bits, i.e. 2 kbit of an 18-kbit primitive. The reported occupancy therefore counts *tiles*, not stored data: roughly 88 % of the allocated block-RAM capacity is unused. Any increase in the input width or in the number of parallel neurons multiplies the tile count directly and exhausts the budget long before depth becomes a constraint.

Timing. All constraints are met at the 125 MHz target (8.000 ns period), as summarized in Table 15: setup WNS 0.038 ns, hold WHS 0.028 ns, pulse width WPWS 3.020 ns, with zero failing endpoints out of 31,828. The corresponding maximum frequency is 125.6 MHz.

For a design whose multipliers are fully pipelined through the DSP input, intermediate and output registers, this figure is lower than one would expect, and the critical path explains why. The worst path does not lie in the arithmetic at all: it runs from a payload register inside the

AXI SmartConnect to the enable input of one of the 192 weight memories, with a data-path delay of 7.298 ns of which 5.974 ns (81.9 %) is routing and only 1.324 ns is logic across seven levels. The limiting factor is thus the physical distribution of the AXI-Lite write path across a large number of small, spatially dispersed memories, which is the same organizational choice identified above as the scaling limit. Consolidating inputs into wider memory words would shorten this path and raise the achievable frequency, in addition to relaxing the capacity bound; we did not pursue it here because 125 MHz is already supplied directly by the PS clock generator and the core latency is not the dominant term in the end-to-end path (Section 6).

Power. Vivado's vectorless estimate for the implemented design is 2.718 W of total on-chip power (2.521 W dynamic, 0.196 W static) at a junction temperature of 56.3 °C. We emphasize that this is a tool estimate obtained by activity propagation rather than a measurement. The breakdown in Table 16 shows that the processing system accounts for 1.535 W, i.e. more than half of the total, while the entire programmable-logic contribution (clocks, logic, signals, block RAM, DSP and I/O) amounts to 0.986 W. Comparisons against accelerator cards should use the latter figure, since those designs do not include an application processor in their reported power.

Table 14: Post-implementation resource utilization on the Xilinx XC7Z020-1CLG484

Resource	Used	Available	Utilization
Slice LUTs	14,445	53,200	27.15%
as logic	14,361	53,200	26.99%
as memory	84	17,400	0.48%
Slice registers	11,372	106,400	10.69%
Block RAM tiles	98.5	140	70.36%
DSP slices	192	220	87.27%
Bonded IOB	14	200	7.00%
BUFGCTRL	1	32	3.13%

Table 15: Post-implementation timing closure at 125 MHz (8.000 ns period, speed grade -1)

Check	Worst slack	Failing endpoints
Setup (WNS)	0.038 ns	0 / 31,828
Hold (WHS)	0.028 ns	0 / 31,828
Pulse width (WPWS)	3.020 ns	0 / 12,091

6.12. Deployment Cost

The platform used here is a Digilent ZedBoard carrying an XC7Z020, a part in the low hundreds of US dollars, against the several thousand typical of a data-center accelerator card. We deliberately avoid a tabulated cost comparison against CPU, GPU and high-end FPGA alternatives, because we have not measured this network on those platforms and a table populated with qualitative labels would add nothing. What the measurements do support is narrower and can be stated directly: a deterministic, sub-microsecond inference path for a network of this size does not require a data-center part. As a point of reference, SparkNoC [13] reaches 337.2 GOP/s at about 7.6 W of compute power on an Arria 10 GX1150 card, an order of magnitude more throughput than the present design delivers, on hardware of a different class and for a throughput-oriented workload rather than single-inference latency.

Table 16: On-chip power, Vivado vectorless estimate for the implemented design

Component	Power
PS7 (processing system)	1.535 W
Signals	0.280 W
Block RAM	0.249 W
DSP	0.236 W
Slice logic	0.150 W
Clocks	0.066 W
I/O	0.005 W
Dynamic total	2.521 W
Device static	0.196 W
Total on-chip	2.718 W
of which PL only	0.986 W

6.13. Run-Time Reconfigurability and Scalability

The framework property claimed in Section 3 is that a single bitstream hosts different networks. This is not a matter of reloading weights alone: the per-layer output widths are held in AXI-Lite configuration registers that the issue FSM reads to derive its iteration counts, so the *shape* of the network is also a run-time parameter.

The verification campaign of Section 6.2 demonstrates this directly. The four networks of Table 8 have different hidden-layer widths, with the first hidden layer ranging from 10 to 64 neurons, yet all four were loaded into the same programmed device over AXI-Lite and each was verified bit-exactly. No re-synthesis, re-implementation or bitstream reload took place between them; loading a network requires $\sum_{\ell} \lceil n_{\ell}/3 \rceil \times 3 \times 64$ weight writes plus the corresponding biases, which for the reference topology is approximately 7800 AXI-Lite transactions.

The limits of this reconfigurability should be stated precisely. The datapath is elaborated for at most four layers, at most 64 neurons per layer and at most 64 inputs per layer; these bounds are compile-time constants of the SIMD array and of the weight-memory instantiation. Any network within those bounds is hosted without re-synthesis; a network exceeding them is not.

Scaling beyond these bounds is limited by memory organization rather than by arithmetic. The design instantiates one memory per (neuron-slot, input) pair, i.e. $\mathrm{PARALLEL_NEURONS} \times \mathrm{MAX_INPUTS} = 192$ weight memories, each of which stores only 128 words and therefore occupies a small fraction of the 18-kbit primitive it is mapped onto. The reported 70.4 % block-RAM occupancy thus reflects the *number* of memories rather than their depth: widening the input vector or increasing the number of parallel neurons increases the block count proportionally and exhausts the budget well before the depth of any individual memory becomes a constraint. Packing several inputs per memory word is the natural way to relax this limit and is left as future work.

7. Discussion

The results support three observations. First, embedding the network in the PL makes the core latency a property of the pipeline rather than of the operating system: the measured core latency is 768 ns and deterministic, whereas the end-to-end host path around it varies from 66 to 476 μs in median across operating systems and shows heavy tails. Second, $Q8.8$ quantization is essentially accuracy-neutral here (at most 0.22 percentage points across five models), so the speed-up does not cost predictive quality. Third, the design fits a low-cost Zynq-7000 at 2.72 W with positive timing slack, so the approach does not depend on premium accelerator hardware. The instrumented per-stage breakdown (Table 12) further shows that only 14.4 μs of the 68.6 μs end-to-end latency is spent on the board, so the main remaining limitation is the host- and network-side software path; moving the Ethernet stack into the PL is the clearest next step.

8. Conclusion

We presented an Ethernet-fed MLP inference framework embedded in the programmable logic of a low-cost Zynq-7000 SoC. Feature vectors are streamed to the PL over AXI-DMA, and a 192-multiplier, 13-stage pipelined core performs the $Q8.8$ inference in 768 ns with deterministic timing, using 27.2 % of the LUTs and 87.3 % of the DSP slices of an XC7Z020 at an estimated 2.72 W of total on-chip power, of which 0.99 W is attributable to the programmable logic. The same network executed bare-metal on the SoC's own Cortex-A9 requires 52.4 μs, so moving the computation into the fabric is worth a factor of 68 against an independent baseline on identical arithmetic (Section 6.10). Decomposed against an earlier sequential implementation of the same network on the same device, the architecture accounts for a factor of 58.7 at a common clock, and 73.4 as the two were actually deployed (Section 6.9).

The property that distinguishes the design from compiler-style FPGA inference frameworks is that the network shape is a run-time rather than an elaboration-time parameter: four networks with different hidden-layer widths were deployed into one unchanged bitstream over AXI-Lite, with no synthesis step between them. Correctness of the deployed arithmetic is not assumed but established: the core was verified bit-exactly against an independent reference model over 400,000 inferences, a process that first exposed and then eliminated three pipeline-alignment defects that neither synthesis nor end-to-end testing had revealed.

Two directions follow from the measured breakdown. First, only 14.4 μs of the 68.6 μs end-to-end latency is spent on the board, so integrating an Ethernet MAC directly in the programmable logic, and thereby removing the host network stack from the path, is the change with the largest remaining effect. Second, the present bounds on network shape (at most four layers of at most 64 neurons and 64 inputs) follow from the one-memory-per-input organization of the weight storage rather than from the arithmetic; packing several inputs per memory word would relax them without additional DSP cost.

CRediT authorship contribution statement

Ahmet Batuhan Özkaner: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing (original draft), Writing (review and editing). *Mustafa Alper Ilık:* Software, Investigation, Writing (review and editing). *Baykal Sarıoğlu:* Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The artifact supporting this study is publicly available at `https://github.com/ozkanerbatuhan/hft` (hardware) and `https://github.com/ozkanerbatuhan/HFT_vitis` (firmware and host tooling). It contains the RTL sources, the self-checking testbench, the cycle-accurate reference model, the host and firmware tooling, the stimulus generator, the post-implementation reports, and the measurement logs behind the latency and verification results. Instructions for rebuilding the bitstream and for reproducing each claim are given in the accompanying documentation.

References

[1] Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

[2] Matteo Aquilina, Eric Budish, and Peter O'Neill. Quantifying the high-frequency trading "arms race". *The Quarterly Journal of Economics*, 137(1):493–564, 2022.

[3] Christian Leber, Benjamin Geib, and Heiner Litz. High frequency trading acceleration using FPGAs. In *Proceedings of the 21st International Conference on Field Programmable Logic and Applications (FPL)*, pages 317–322, 2011.

[4] John W. Lockwood and Madhu Monga. Implementing ultra low latency data center services with programmable logic. In *Proceedings of the IEEE 23rd Annual Symposium on High-Performance Interconnects (HOTI)*. IEEE, 2015.

[5] Sunil Puranik, Mahesh Barve, Swapnil Rodi, and Rajendra Patrikar. Acceleration of trading system back end with FPGAs using high-level synthesis flow. *Electronics*, 12(3):520, 2023.

[6] Nick Brown, Mark Klaisoongnoen, and Oliver Thomson Brown. Optimisation of an FPGA credit default swap engine by embracing dataflow techniques. In *IEEE International Conference on Cluster Computing (CLUSTER), FPGA for HPC Workshop*, 2021.

[7] Chen Zhang, Peng Li, Guangyu Sun, Yijin Guan, Bingjun Xiao, and Jason Cong. Optimizing FPGA-based accelerator design for deep convolutional neural networks. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 161–170, 2015.

[8] Jiantao Qiu, Jie Wang, Song Yao, Kaiyuan Guo, Boxun Li, Erjin Zhou, Jincheng Yu, Tianqi Tang, Ningyi Xu, Sen Song, Yu Wang, and Huazhong Yang. Going deeper with embedded FPGA platform for convolutional neural network. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 26–35, 2016.

[9] Kaiyuan Guo, Lingzhi Sui, Jiantao Qiu, Jincheng Yu, Junbin Wang, Song Yao, Song Han, Yu Wang, and Huazhong Yang. Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(1):35–47, 2018.

[10] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. FINN: A framework for fast, scalable binarized neural network inference. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 65–74, 2017.

[11] Michaela Blott, Thomas B. Preußer, Nicholas J. Fraser, Giulio Gambardella, Kenneth O'Brien, Yaman Umuroglu, Miriam Leeser, and Kees Vissers. FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. *ACM Transactions on Reconfigurable Technology and Systems*, 11(3):1–23, 2018.

[12] Javier Duarte, Song Han, Philip Harris, Sergo Jindariani, Edward Kreinar, Benjamin Kreis, Jennifer Ngadiuba, Maurizio Pierini, Ryan Rivera, Nhan Tran, and Zhenbin Wu. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation*, 13(07):P07027, 2018.

[13] Ming Xia, Zunkai Huang, Li Tian, Hui Wang, Victor Chang, Yongxin Zhu, and Songlin Feng. SparkNoC: An energy-efficiency FPGA-based accelerator using optimized lightweight CNN for edge computing. *Journal of Systems Architecture*, 115:101991, 2021.

[14] Vasileios Tsoutsouras, Konstantina Koliogeorgi, Sotirios Xydis, and Dimitrios Soudris. An exploration framework for efficient high-level synthesis of support vector machines: Case study on ECG arrhythmia detection for Xilinx Zynq SoC. *Journal of Signal Processing Systems*, 88(2):127–147, 2017.

[15] Teng Wang, Chao Wang, Xuehai Zhou, and Huaping Chen. A survey of FPGA based deep learning accelerators: Challenges and opportunities. *arXiv preprint arXiv:1901.04988*, 2019.

[16] Antonio Rios-Navarro, Ricardo Tapiador-Morales, Angel Jimenez-Fernandez, Manuel Dominguez-Morales, Carlos Amaya, and Alejandro Linares-Barranco. Performance evaluation over HW/SW co-design SoC memory transfers for a CNN accelerator. In *Proceedings of the IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, 2018.

[17] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.

[18] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *Journal of Machine Learning Research*, 18(1):6869–6898, 2017.

[19] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*, 2021.

[20] Thierry Moreau, Tianqi Chen, Luis Vega, Jared Roesch, Eddie Q. Yan, Lianmin Zheng, Josh Fromm, Ziheng Jiang, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. A hardware–software blueprint for

flexible deep learning specialization. *IEEE Micro*, 39(5):8–16, 2019.

[21] Yunxuan Yu, Chen Wu, Tiandong Zhao, Kun Wang, and Lei He. OPU: An FPGA-based overlay processor for convolutional neural networks. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 28(1):35–47, 2020.

[22] Yunxuan Yu, Tiandong Zhao, Kun Wang, and Lei He. Light-OPU: An FPGA-based overlay processor for lightweight convolutional neural networks. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 122–132, 2020.

[23] AMD. Vitis AI development environment. Product documentation, 2024.

[24] Yaman Umuroglu, Yash Akhauri, Nicholas J. Fraser, and Michaela Blott. LogicNets: Co-designed neural networks and circuits for extreme-throughput applications. In *Proceedings of the 30th International Conference on Field Programmable Logic and Applications (FPL)*, pages 291–297, 2020.

[25] Marta Andronic and George A. Constantinides. PolyLUT: Learning piecewise polynomials for ultra-low latency FPGA LUT-based inference. In *Proceedings of the International Conference on Field Programmable Technology (ICFPT)*, 2023.

[26] Marta Andronic and George A. Constantinides. NeuraLUT: Hiding neural network density in boolean synthesizable functions. In *Proceedings of the 34th International Conference on Field-Programmable Logic and Applications (FPL)*, 2024.

[27] Zihao Zhang, Stefan Zohren, and Stephen Roberts. DeepLOB: Deep convolutional neural networks for limit order books. *IEEE Transactions on Signal Processing*, 67(11):3001–3012, 2019.

[28] Adamantios Ntakaris, Martin Magris, Juho Kanniainen, Moncef Gabbouj, and Alexandros Iosifidis. Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*, 37(8):852–866, 2018.